

Chapter 6: QUEUE (FIFO)

Queue is one of the most important STL containers.

If Stack is:

```
Last In First Out (LIFO)
```

Then Queue is:

```
First In First Out (FIFO)
```

Real Life Example

Think of people standing in a ticket line.

Front

10

20

30

40

Back

Person 10 came first.

So 10 leaves first.

Header

```
#include <queue>
```

Declaration

```
queue<int> q;
```

Basic Operations

1. push()

Insert at back.

```
q.push(10);  
q.push(20);  
q.push(30);
```

Queue:

Front

10 20 30

Back

Complexity

$O(1)$

2. pop()

Removes front element.

```
q.pop();
```

Before:

10 20 30

After:

```
20 30
```

Complexity

```
O(1)
```

3. front()

Returns first element.

```
cout << q.front();
```

Output:

```
10
```

4. back()

Returns last element.

```
cout << q.back();
```

Output:

```
30
```

5. size()

```
cout << q.size();
```

Output:

```
3
```

6. empty()

```
q.empty();
```

Returns:

```
true  
false
```

Complete Example

```
#include <iostream>
#include <queue>
using namespace std;

int main()
{
    queue<int> q;

    q.push(10);
    q.push(20);
    q.push(30);

    cout << q.front() << endl;
    cout << q.back() << endl;

    q.pop();

    cout << q.front();
}
```

Output:

```
10  
30
```

20

Printing Queue

Queue has:

✗ No iterators

✗ No indexing

So use copy.

```
queue<int> temp = q;

while(!temp.empty())
{
    cout << temp.front() << " ";
    temp.pop();
}
```

Swap Queue

```
q1.swap(q2);
```

Complexity:

```
O(1)
```

Emplace

```
q.emplace(10);
```

Same idea as stack.

Time Complexity

Operation	Complexity
push	$O(1)$
pop	$O(1)$
front	$O(1)$
back	$O(1)$
size	$O(1)$
empty	$O(1)$

Queue vs Stack

Stack	Queue
LIFO	FIFO
top()	front()
push()	push()
pop() removes top	pop() removes front

Example:

Stack:

```
1 2 3  
  
Pop -> 3
```

Queue:

```
1 2 3  
  
Pop -> 1
```

Interview Question 1

Generate Binary Numbers

Input:

5

Output:

1
10
11
100
101

Uses Queue.

Interview Question 2

First Non-Repeating Character

Input:

aabc

Output:

a -1 b b

Uses:

queue + frequency array

Most Important Use: BFS

Breadth First Search

Graph:

```
1 -- 2
|    |
3 -- 4
```

Traversal:

```
1 2 3 4
```

Uses Queue.

BFS Template

```
queue<int> q;

q.push(start);

while(!q.empty())
{
    int node = q.front();
    q.pop();

    for(auto child:adj[node])
    {
        q.push(child);
    }
}
```

You will use this constantly in Graphs.

Queue of Pairs

Very common.

```
queue<pair<int,int>> q;
```

Example:

```
q.push({1,2});
```



```
q.push({3,4});
```

Access:

```
cout << q.front().first;  
cout << q.front().second;
```

Queue of Vectors

Possible:

```
queue<vector<int>> q;
```

Rare.

Queue of Custom Objects

```
class Student  
{  
public:  
    int id;  
};  
  
queue<Student> q;
```

Interview level.

Practice Questions

Easy

Q1

Push:

```
10 20 30
```

Print front and back.

Q2

Pop one element.

Print front.

Q3

Print queue size.

Q4

Print all queue elements.

Q5

Copy one queue into another.

Medium

Q6

Reverse first K elements of queue.

Input:

```
1 2 3 4 5
K=3
```

Output:

```
3 2 1 4 5
```

Hint:

```
stack + queue
```

Q7

Reverse entire queue.

Q8

Find sum of all queue elements.

Q9

Generate first N binary numbers.

Q10

First non-repeating character stream.

Interview Level

Q11

Implement Stack using Queue

LeetCode #225

Q12

Implement Queue using Stack

LeetCode #232

Q13

Rotting Oranges

LeetCode #994

(BFS)

Q14

01 Matrix

LeetCode #542

(BFS)

Q15

Word Ladder

LeetCode #127

(BFS)

Common Mistakes

Mistake 1

```
q.front();
```

on empty queue

✗ Runtime Error

Always check:

```
if(!q.empty())
```

Mistake 2

```
q.pop();
```

on empty queue

✗ Runtime Error

Mistake 3

```
q[0]
```

✗ Not Allowed

Queue has no indexing.

Revision Sheet

Header

```
#include <queue>
```

Declaration

```
queue<int> q;
```

Insert

```
q.push(x);  
q.emplace(x);
```

Remove

```
q.pop();
```

Access

```
q.front();  
q.back();
```

Utilities

```
q.size();  
q.empty();  
q.swap();
```

Complexity

```
push  -> O(1)  
pop   -> O(1)
```

```
front -> O(1)
back  -> O(1)
size  -> O(1)
```

Queue Mental Trigger

When you see:

- BFS
- Level Order Traversal
- Waiting Line
- First Come First Serve
- Shortest Path (Unweighted Graph)
- Multi-source BFS

Think:

QUEUE

Assignment

Solve:

1. Reverse Queue.
 2. Reverse First K Elements.
 3. Generate Binary Numbers.
 4. First Non-Repeating Character.
 5. Implement Stack Using Queue.
 6. Implement Queue Using Stack.
 7. BFS Traversal.
 8. Level Order Traversal of Binary Tree.
-
-
-

DEQUE (Double Ended Queue)

Before learning Set/Map, learn Deque because many STL containers internally use it.

What is Deque?

Deque = Double Ended Queue

You can insert and delete from:

- Front ☒
- Back ☒

Queue allows:

Push Back
Pop Front

Deque allows:

Push Front
Push Back

Pop Front
Pop Back

Header

```
#include <deque>
```

Declaration

```
deque<int> dq;
```

Visualization

Front

10 20 30

Back

1. push_back()

Insert at end.

```
dq.push_back(10);  
dq.push_back(20);  
dq.push_back(30);
```

Result:

```
10 20 30
```

Complexity:

```
O(1)
```

2. push_front()

Insert at front.

```
dq.push_front(5);
```

Result:

```
5 10 20 30
```

Complexity:

```
O(1)
```

3. pop_back()

Remove last element.


```
dq.pop_back();
```

Before:

```
5 10 20 30
```

After:

```
5 10 20
```

Complexity:

```
O(1)
```

4. pop_front()

Remove first element.

```
dq.pop_front();
```

Before:

```
5 10 20
```

After:

```
10 20
```

Complexity:

```
O(1)
```

5. front()

Returns first element.

```
cout << dq.front();
```

Output:

```
10
```

6. back()

Returns last element.

```
cout << dq.back();
```

Output:

```
20
```

7. size()

```
cout << dq.size();
```

8. empty()

```
dq.empty();
```

Returns:

```
true  
false
```

Indexing Support

Unlike Queue and Stack:

```
dq[0]  
dq[1]  
dq[2]
```

Works.

Example:

```
cout << dq[1];
```

at()

Safer access.

```
dq.at(2);
```

Iterators

Supported.

```
for(auto it=dq.begin(); it!=dq.end(); it++)  
{  
    cout<<*it<<" ";  
}
```

Range Loop

```
for(auto x:dq)
{
    cout<<x<<" ";
}
```

insert()

Insert anywhere.

```
deque<int> dq={1,2,4};

dq.insert(dq.begin()+2,3);
```

Result:

```
1 2 3 4
```

Complexity:

```
O(n)
```

erase()

Remove element.

```
dq.erase(dq.begin()+2);
```

clear()

Remove all elements.

```
dq.clear();
```

resize()

```
dq.resize(10);
```

swap()

```
dq1.swap(dq2);
```

Assign

```
dq.assign(5,100);
```

Result:

```
100 100 100 100 100
```

Time Complexity Table

Operation	Complexity
push_front	O(1)
push_back	O(1)
pop_front	O(1)
pop_back	O(1)
front	O(1)
back	O(1)
[]	O(1)
insert middle	O(n)
erase middle	O(n)

Vector vs Deque

Feature	Vector	Deque
Push Back	$O(1)$	$O(1)$
Pop Back	$O(1)$	$O(1)$
Push Front	$O(n)$	$O(1)$
Pop Front	$O(n)$	$O(1)$
Random Access	$O(1)$	$O(1)$

When to Use Deque?

Use when:

Need:

```
Insert Front  
Insert Back  
  
Delete Front  
Delete Back
```

Frequently.

Interview Question 1

Sliding Window Maximum

Input:

```
1 3 -1 -3 5 3 6 7  
  
K=3
```

Output:

```
3 3 5 5 6 7
```

Optimal solution uses:

deque

Complexity:

$O(n)$

Very famous.

Interview Question 2

Design Circular Deque

LeetCode #641

Uses deque concept.

Practice Questions

Easy

Q1

Insert:

10 20 30

using push_back.

Print.

Q2

Insert:

5

at front.

Q3

Remove front.

Print deque.

Q4

Remove back.

Print deque.

Q5

Print first and last element.

Medium

Q6

Reverse deque.

Hint:

```
reverse()
```

Q7

Insert 100 at index 2.

Q8

Delete element at index 3.

Q9

Find maximum element.

Hint:

```
max_element()
```

Q10

Rotate deque by 1.

Input:

```
1 2 3 4 5
```

Output:

```
5 1 2 3 4
```

Interview Level

Q11

Sliding Window Maximum

LeetCode #239

Q12

Shortest Subarray With Sum At Least K

LeetCode #862

Q13

Design Circular Deque

LeetCode #641

Common Mistakes

Mistake 1

```
dq.front();
```

on empty deque

✖ Error

Mistake 2

```
dq.pop_front();
```

on empty deque

✖ Error

Mistake 3

Thinking deque == queue

✖ Wrong

Deque supports:

```
push_front()  
pop_front()  
push_back()  
pop_back()
```

Queue does not.

Revision Sheet

Header

```
#include <deque>
```

Declaration

```
deque<int> dq;
```

Front Operations

```
push_front()  
pop_front()  
front()
```

Back Operations

```
push_back()  
pop_back()  
back()
```

Access

```
[]  
at()
```

Utilities

```
size()  
empty()  
clear()  
resize()  
swap()  
assign()
```

Iterators

```
begin()  
end()  
rbegin()  
rend()
```

Complexity

```
push_front -> O(1)  
push_back  -> O(1)  
pop_front  -> O(1)
```

```
pop_back    -> O(1)
access      -> O(1)
```

Very Important Transition

Until now you've learned:

✓ Pair ✓ Vector ✓ Iterators ✓ Algorithms ✓ Stack ✓ Queue ✓ Deque

Now we enter the **Hashing & Searching STL**, which is the most important part for interviews.

These containers solve:

```
Find duplicate
Count frequency
Unique elements
Fast searching
Fast insertion
```